The Complete Guide to Specifying Work for AI: From "Black Magic" to Engineering Discipline

A Practitioner's Reference

Hayen Mill & Henrique Jr. Sanchez
Revised with feedback of Cory Gwin
March 2026

Executive Summary

In mid-2025, Cory Gwin (an engineer on GitHub Copilot) wrote that "writing good specs is still black magic. There's no reliable formula." By late 2025, that statement had become the central challenge of AI-assisted software development. The METR randomized controlled trial found that experienced developers using AI tools were actually 19% slower than those working without them, while believing they were 20% faster. Part of that gap is a specification gap, but part of it is a model reliability gap that no specification format has yet solved.

This document compiles the frameworks, templates, and empirical findings available as of early 2026 for defining work for AI systems. It synthesizes research from Anthropic, Thoughtworks, Martin Fowler's team, Google's Agent Development Kit, the Manus AI team, LangChain, METR, and practitioners building production systems.

The recurring finding across all sources is this: **specification quality is the highest-leverage factor within the developer's direct control.** The model is often not the bottleneck, but sometimes it is. Hallucination, non-determinism, and cognitive overhead are real problems that better specs alone cannot solve. What better specs *can* do is eliminate the largest category of preventable failure: the AI executing competently on a misunderstood instruction. This guide exists to systematically close that gap, while being honest about what remains unsolved.

How to Use This Document

This guide is organized for practitioners. Part 1 provides the practical tools: templates, checklists, and patterns you can use immediately. Parts 2–5 provide the supporting frameworks: the evidence base, correctness definitions, spec formats, and the role of tests as specification. Parts 6–8 cover context engineering, anti-patterns, and full reference templates. If you need to

specify a task right now, start with Part 1. If you want to understand why these tools work, read Parts 2–5.

Part 1: Quick Start — The Practical Tools

These are the immediately actionable tools from this guide. Each one addresses a specific, common failure mode in AI-assisted work.

1.1 The Quick-Reference Decision Framework

Before any AI-assisted task, answer three questions:

1. **What mode?** Assistive (you type, AI suggests) / Directed (you describe, AI types, you review) / Autonomous (you assign, AI executes, you review the PR) / Continuous (AI runs as a background service)
2. **What size spec?** Intent Doc for small tasks / Minimal Viable Spec for medium tasks / Full PRD for large tasks
3. **What are the boundaries?** What must the AI NOT do?

As autonomy increases, spec rigor must increase proportionally. In Assistive mode, your brain is the spec. In Autonomous mode, the written spec is the only control mechanism between intent and execution. This table summarizes the relationship:

Mode	Human Understanding	Risk of Incorrect Output	Spec Rigor Required
Assistive	Very High	Very Low	Low
Directed	High	Low–Medium	Medium
Autonomous	Medium	Medium–High	High
Autonomous + Spec	Medium	Medium (reduced)	High (by definition)
Continuous	Low	High	Very High

The "Autonomous + Spec" row is the key finding from Gwin's analysis: adding a spec is the only intervention in the table that improves a risk rating.[1]

1.2 The Intent Doc (The 4-Line Spec)

Use this for any task that carries risk of misinterpretation. It takes 30 seconds to write and prevents the most common category of AI failure: acting on a misunderstood instruction.

1. TASK: [What specifically was asked for]
2. SCOPE: [Who/what is affected, who/what is excluded]
3. BOUNDARIES: [What the AI will NOT do]
4. IF UNCERTAIN: [The error-handling protocol: ask, flag, or stop]

Example for a code refactoring task:

1. TASK: Extract the validation logic from UserController.create() into a separate UserValidator class.
2. SCOPE: Only the create() method. Do not modify update() or delete(). The new class goes in src/validators/.
3. BOUNDARIES: Do not change any test files. Do not modify the API response format. Do not add new dependencies.
4. IF UNCERTAIN: If a validation rule seems to also apply to update(), add a TODO comment rather than extracting it.

1.3 The Minimal Viable Spec

For tasks where the Intent Doc is too sparse and a full PRD is too heavy:

```
## Task: [One-sentence description]

### Goal
[What this should accomplish, written for the audience who will use
the output]

### Acceptance Criteria
- [ ] [Binary-testable condition 1]
- [ ] [Binary-testable condition 2]
- [ ] [Binary-testable condition 3]

### Constraints
- Follow the patterns in [specific file or directory]
- Technology: [specific stack/tools]
- Performance: [specific budget if applicable]

### Non-Goals
- [Explicit exclusion 1]
- [Explicit exclusion 2]
```

If Uncertain
[Guess / Flag / Stop — pick one and explain when it applies]

1.4 The Vague Requirements Translator

Stakeholders and developers often give fuzzy instructions. A spec must translate these into observable, testable behaviors.

Vague Requirement	Why It Fails	Spec Translation
"Make it better"	No measurable criterion	"Section 2 must be under 200 words and contain no adjectives."
"Reduce hallucinations"	No definition of "hallucination"	"If a source cannot be cited for a factual claim, the claim must be omitted."
"Make it accurate"	No definition of "accuracy"	"All numerical values must match the source spreadsheet to two decimal places."
"Improve the UX"	No definition of "improvement"	"The primary action button must be reachable within one click from the landing page."
"Make it production-ready"	No definition of "production"	"The code must include error handling for all external API calls, have >80% test coverage, and pass the existing linter configuration."
"Keep it simple"	No definition of "simple"	"The implementation must use no more than 3 files, no external dependencies beyond those in package.json, and no function longer than 30 lines."

The principle: for every qualitative requirement, identify the specific observable behavior that would satisfy it and express that behavior in measurable terms. If you cannot perform this translation, the requirement is not yet ready to be specified.

1.5 The Complete Spec Checklist

Use this to audit any specification before execution:

- ☐ Problem statement precedes any solution description
- ☐ Audience for the output is explicitly named
- ☐ Success metrics are quantitative and binary-testable
- ☐ Failure modes are enumerated
- ☐ Danger modes are enumerated (if applicable)

- ☐ Non-goals/boundaries are at least as detailed as goals
- ☐ All technical constraints are explicit (stack, patterns, conventions)
- ☐ Uncertainty policy is stated (guess vs. flag vs. stop)
- ☐ Priority tradeoffs are stated (when X conflicts with Y, prefer ...)
- ☐ Verification steps are described
- ☐ All vague terms have measurable translations
- ☐ No references to tacit knowledge without providing the knowledge itself
- ☐ Abstraction level is appropriate for the task size
- ☐ A different person (or AI) could execute this unambiguously

Score: 11–14 pass = ready to execute. 7–10 = revise weak areas. Below 7 = fundamental rethink needed.

The tools above work immediately. Parts 2–5 explain why they work and how to handle harder cases — start with Part 3 if your task involves any ambiguity about what "correct" means.

Part 2: The Evidence — Why Specification Matters (and What It Can't Fix)

2.1 The Empirical Evidence

The case for specification rigor goes beyond mere theory. Several independent lines of evidence converge, but they converge on a more nuanced conclusion than "specs fix everything."

The METR RCT (July 2025). METR conducted a randomized controlled trial with 16 experienced open-source developers completing 246 real tasks in mature repositories they had an average of 5 years of experience with. Developers using frontier AI tools (Cursor Pro with Claude 3.5/3.7 Sonnet) were 19% slower than those working without AI.[2] Developers expected AI to speed them up by 24%, and even after experiencing the slowdown, they still believed AI had sped them up by 20%.[2]

The study identified five contributing factors: overoptimism about AI usefulness leading to overuse, high familiarity with repositories making AI less additive, low AI reliability requiring extensive output validation, extra cognitive load from context-switching between coding and prompting, and implicit quality requirements (documentation, testing, linting) that AI struggled with. Of these five factors, only the last (implicit quality requirements) maps directly to specification. The others reflect model reliability, cognitive overhead, and workflow issues that better specs alone cannot address. The METR study is evidence that AI-assisted development has multiple unsolved problems, of which specification is one.

Thoughtworks on Spec-Driven Development (December 2025). Thoughtworks' Liu Shangqi observed that code generated from structured technical specifications was significantly higher quality than code generated from plain-text requirements. They found that semi-structured inputs improved reasoning performance and reduced hallucinations.[3] The Thoughtworks analysis was careful to note this as a qualified improvement, that specs "can help reduce" hallucinations but not eliminate them, and acknowledged that LLMs do not generate deterministic code like traditional code generation. Their analysis drew a parallel to behavior-driven development (BDD), noting that Given/When/Then scenarios and clear structure remain valid in the AI era.[3]

Martin Fowler's Team on SDD Tools (October 2025). Birgitta Böckeler evaluated three spec-driven development tools (Kiro, GitHub spec-kit, and Tessl) and found that lower-abstraction specs reduced the number of interpretive steps an LLM must take, and therefore reduced error rates.[4] She identified three levels of SDD: spec-first (write a spec, use it, discard it), spec-anchored (keep the spec as a living document), and spec-as-source (the spec is the primary artifact; humans never edit code directly).

Critically, Böckeler also warned that SDD tools may create a "false sense of control." She wondered if some tools were trying to feed AI agents existing workflows too literally, ultimately amplifying challenges like review overload and hallucinations, and raised the possibility of "Verschlimmbesserung": making things worse in the attempt to make them better.[4] Agents frequently did not follow all instructions, and larger context windows did not guarantee compliance. Böckeler observed agents both ignoring instructions and exceeding them by following instructions "too avidly."[4] This finding is essential context for the rest of this guide: specifications improve outcomes on average, but they do not guarantee them.

Red Hat on Feedback Loops (October 2025). Red Hat's analysis found that spec-plus-lessons-learned feedback loops reduce agent errors over time. By maintaining a record of where specs failed and feeding corrections back into future specifications, teams could build institutional knowledge about what their AI systems consistently misunderstand.

2.2 The Core Philosophical Shift

The evidence points to a fundamental reframing. Anthropic's engineering team (September 2025) described the shift from "prompt engineering" to "context engineering": building with LLMs is less about finding the right words and more about configuring the right context.[5] Google's ADK team extended this to "context as compilation", treating context as a computed view over structured state, not an accumulated conversation.[6] The most mature practitioners describe their work as "orchestration": decomposing fuzzy goals into concrete workflows where each step has defined inputs, outputs, and correctness criteria.

2.3 The Cost Structure Inversion

The cost of writing code is dropping fast. When implementation was expensive, the optimal strategy was to get it right the first time. Now, when implementation is cheap, you can generate

multiple versions and pick the best one. This inverts where quality control lives: away from code review for every line and toward tests, type checkers, and linters that verify behavior automatically, with human review focused on what automated tools cannot assess: security implications, compliance, architectural fit, and judgment.

2.4 Mode Fluency

Gwin observes that productive developers switch modes two or three times on a single change: open an issue for the agent (Autonomous), review and test locally (Directed), then drop into Assistive mode to fix final issues.[1] The METR study's finding may be partly explained by mode rigidity: developers who stayed in a single mode were slower than those who fluently switched based on what the task demanded.

Part 3: Defining Correctness Before Writing Anything

Before writing any spec, you must answer a prior question: what constitutes a correct output? Most systems fail because this definition is missing. The spec describes *what to do*; the correctness contract defines *what "done well" looks like*.

3.1 The Correctness Discovery Framework

Use these six questions to establish the correctness contract for any task before you begin specifying it. They are ordered from most concrete to most abstract, and each constrains the specification you will write.

1. Audience: Who uses this output and what decision will they make based on it?

A utility function used by junior developers who won't read the internals needs a clear signature and docstring. The same logic used as an internal helper can be terser. Specifying the audience prevents the AI from optimizing for the wrong consumer.

Example: "The audience is a junior developer who will use this function without reading its internals. The function signature and docstring are therefore more important than the implementation."

2. Failure Definition: What would make this output useless?

This is not the inverse of success, but a separate question. Success has many dimensions; failure often has specific, predictable shapes.

Example: "This output is useless if it contains placeholder text, if it references APIs that do not exist in our codebase, or if it introduces a dependency not in our package.json."

3. Danger Definition: What would make this output actively harmful?

Some outputs are not merely useless but dangerous. A migration script that silently drops a column, or an auth change that bypasses rate limiting, is not just wrong but can be hazardous. This question forces explicit consideration of the worst-case scenario.

Example: "This output is dangerous if it exposes internal API keys, if it modifies the production database schema, or if it silently changes existing test assertions to make failing tests pass."

4. Uncertainty Policy: What should the AI do when uncertain?

LLMs do not have a reliable internal sense of their own uncertainty. You can instruct them on what to do in situations you know will be ambiguous. The two poles are: guess (useful when speed matters more than precision) and refuse/flag (useful when precision matters more than speed).

Example: "If you are unsure which of two architectural patterns to use, describe both options and ask for clarification. Do not pick one silently."

5. Risk Tolerance: What is worse, a confident wrong answer or a refusal to answer?

In a brainstorming session, a confident wrong answer is cheap and may spark useful thinking. In a compliance review, a confident wrong answer could have legal consequences.

Example: "A refusal to answer is always preferable to an incorrect answer. If you cannot determine the correct return type, leave a TODO comment rather than guessing."

6. Verification: How would you check if the output is correct?

If you cannot describe how to verify an output, you cannot write a useful spec. This question also reveals hidden assumptions: if verification requires information you have not provided to the AI, you have found a gap in your spec.

Example: "The output is correct if all existing tests pass, if the new endpoint returns a 200 status on the provided sample inputs, and if `npm run lint` produces no new warnings."

3.2 The Tradeoff Forcer

You cannot maximize all values simultaneously. A good spec forces a choice between competing priorities. If you do not force this choice explicitly, the AI will make it implicitly (and often incorrectly), because LLMs are trained to be helpful and will try to satisfy every constraint simultaneously. When constraints conflict, the model resolves the conflict unpredictably.

The key tradeoffs to address: Boldness vs. Caution, Completeness vs. Speed, Precision vs. Coverage, Readability vs. Performance, and Consistency vs. Correctness (matching existing patterns vs. following best practices). For each, the spec should include a statement like: "When

forced to choose between X and Y, prefer X." Making tradeoffs explicit transforms an implicit, uncontrolled choice into an explicit, controlled one. **This is one of the highest-leverage specification techniques available**.

Part 4: Spec Formats by Task Size

The most common specification failure is a mismatch between spec effort and task complexity. Over-specifying small tasks wastes time. Under-specifying large tasks produces errors.

4.1 Matching Spec to Task

Small tasks (bug fixes, single-function changes): Use the Intent Doc (Part 1.2). Full SDD workflows add overhead that exceeds the task cost. Böckeler found that when she asked Kiro to fix a small bug, the requirements workflow turned it into 4 "user stories" with 16 acceptance criteria, basically like bringing a sledgehammer for a nut.[4]

Medium tasks (features, multi-file changes): Use the Minimal Viable Spec (Part 1.3) with the hybrid behavior/contract format shown in Section 4.3.

Large tasks (new systems, major refactors): Use the full PRD structure (Section 4.2). This is where the specification investment pays for itself, because the coordination cost is high enough to justify it.

Ongoing systems (continuous AI, background agents): Spec-anchored or spec-as-source. The specification must encode organizational policy and architectural principles, not just task-level instructions. Böckeler's analysis of spec-first, spec-anchored, and spec-as-source levels provides a useful taxonomy of how persistent the spec should be for your use case.[4]

4.2 The Product Requirements Document (PRD)

The PRD bridges product intent and engineering execution. Each section exists for a specific reason, and the ordering is not arbitrary, with later sections depending on earlier ones.

1. What & Why. The outcome and the justification. Everything in the spec should trace back to this section.

2. Success Metrics. Measurable indicators of value. Metrics must be quantitative: not "improve experience" but "reduce average task completion time from 45 seconds to 30 seconds." If you cannot specify a metric, the requirement may be too vague to implement.

3. User Stories. The user's perspective, in domain language. Use Given/When/Then format where precision matters: "GIVEN a logged-in user with admin privileges, WHEN they click

'Delete Account,' THEN the system must display a confirmation dialog with a 30-second countdown."

4. Acceptance Criteria. Testable conditions for completion. Every criterion must be binary: either it passes or it fails. "QA can verify pass/fail" is the litmus test.

5. Non-Goals. This is the single highest-leverage section in a PRD. If you do not say something is out of scope, stakeholders (and AI agents) will assume it is in scope. Scope creep via omission is the most common failure mode in AI-assisted development. Be aggressively specific: "Non-goals: mobile responsiveness, internationalization, dark mode, accessibility beyond WCAG 2.1 AA, support for browsers other than Chrome and Firefox."

6. Constraints. Technology stack, coding standards, performance budgets, security requirements, existing patterns to follow, external dependencies. Constraints narrow the solution space, which directly reduces the number of interpretive steps the AI must take.

7. Open Questions. Documenting what you do not know is as important as documenting what you do know. Open questions signal that certain areas require additional information before implementation. An AI that encounters an open question should flag it rather than guess, but only if the spec tells it to do so.

4.3 The Specification as a Structured Artifact

Thoughtworks emphasizes that a specification should explicitly define external behavior: input/output mappings, preconditions/postconditions, invariants, constraints, interface types, and integration contracts.[3] While LLMs allow natural-language descriptions, structured and semi-structured inputs still significantly improve reasoning performance. The lesson: use natural language for *describing* behavior, but use structured formats for *constraining* it. A practical hybrid format:

```
## Feature: User Authentication

### Behavior Specification
GIVEN a user with valid credentials
WHEN they submit the login form
THEN the system returns a JWT token with a 24-hour expiry
AND sets an HttpOnly cookie
AND logs the authentication event

### Interface Contract
POST /api/auth/login
Request: { email: string, password: string }
Response 200: { token: string, expiresAt: ISO8601 }
Response 401: { error: "invalid_credentials" }
Response 429: { error: "rate_limited", retryAfter: number }
```

Constraints
- Password attempts limited to 5 per 15-minute window per IP
- Tokens must use RS256 signing
- Must integrate with existing middleware in src/middleware/auth.ts

4.4 Separating Functional from Technical Specs

Functional specs describe *what* the software should do from the user's perspective. Technical specs describe *how* the software should do it from the system's perspective. Separating them means you can update one without disturbing the other, and you can use different prompting strategies: functional specs benefit from natural language, while technical specs benefit from structured formats.

4.5 Micro-Specs and Orchestration Patterns

For daily interactions, macro-level documents are too slow. These lightweight patterns provide specification discipline at the speed of individual tasks.

The Contract-First Pattern. Before the AI executes any multi-step task, establish a mutual agreement on the plan. Step 1 (Intent): You describe what you want. Step 2 (Check): The AI mirrors its understanding back to you, including its interpretation of ambiguous points. Step 3 (Receipt): You confirm, correct, or refine. Only then does execution begin. This costs one extra round-trip but prevents the single most expensive category of AI failure: executing a multi-hour plan based on a misunderstood premise.

Decomposed Prompting. Complex tasks must be broken into sub-specs, each with its own correctness criteria. For any task involving more than one logical step: decompose into sequential sub-tasks with defined inputs and outputs, specify correctness criteria for each sub-task independently, and execute sequentially, verifying each output before passing it as input to the next step. Each sub-task is a mini-spec. The technique directly reduces hallucination by limiting what the AI must "figure out" at any one step.

The Autonomous Looping Pattern. For autonomous-mode work, include: entry criteria (what inputs the agent has), success criteria (must be machine-verifiable), iteration budget (how many attempts before escalating, since without this, agents loop could indefinitely), escalation protocol (what to do when stuck), and side-effect boundaries (what the agent may and may not modify).

Part 5: The Verification Gap — Tests as Specification

Prose specifications improve AI output quality on average, but they do not eliminate variance. Böckeler found that the same spec generated different code on repeated runs. Agents frequently did not follow all instructions even with elaborate templates.[4] This is the central unsolved problem of AI-assisted development, and this guide would be dishonest to treat it as a footnote.

5.1 The Non-Determinism Problem

Every practical technique in this guide, from Intent Docs, correctness contracts, structured specs, and non-goals improves the *probability* of good output, but none of them *guarantee* it. The reason is, of course, fundamental: LLMs are non-deterministic. Identical prompts produce varying outputs. Larger context windows do not guarantee that the model will attend to everything in the context. Templates and workflows create a *feeling* of control without guaranteeing it.

This means that any approach to AI specification that relies solely on prose descriptions of intent is structurally incomplete. You can write the most precise natural-language spec imaginable, and the AI may still produce output that violates it, and not because the spec was bad, but because the model's attention or interpretation varied on that particular run.

5.2 The Pattern That Works: Specs + Tests + Iteration

The most promising pattern to emerge from practice combines prose specifications with executable tests and iteration loops. The prose spec provides intent, context, and constraints. The tests provide the machine-verifiable correctness contract. The iteration loop catches non-deterministic failures mechanically.

Cloudflare's vinext project provides a concrete example. Almost every line of code in vinext was written by AI, but every line passes the same quality gates you'd expect from human-written code: 1,700+ Vitest tests, 380 Playwright E2E tests, full TypeScript type checking, and linting, all running on every pull request.[9] This included tests ported directly from the Next.js test suite, which functioned as a machine-readable specification, so the AI didn't need to understand intent, it implemented against a contract directly.

The process started with a plan, a couple of hours with Claude to define the architecture. From there, the workflow was: define a task, let the AI write the implementation and tests. Then: verification, like running the test suite and Iteration, where if tests fail, feed the error logs back to the AI for correction. The human engineer focused on architecture and correcting dead-end logic.[9]

The critical insight from the vinext case is that comprehensive test suites plus capable models make reimplementation costs approach zero. The test suite becomes the primary quality gate, instead of human code review.

5.3 What Tests Can and Cannot Catch

This pattern is powerful but not complete. Most of the tests driving vinext were functional requirements such as "make it behave like X," "match these outputs," "pass these cases." Vulnerabilities do not live there. They live in the negative space: in complex interactions between layers, in undocumented heuristics, in the things nobody wrote a test for. The model's objective is to pass the tests, and it will take whatever route gets it there.

Cloudflare's vinext was released with critical security flaws almost immediately after launch[11] because the functional tests passed, but security-relevant behaviors that existed as undocumented heuristics in Next.js (like automatically excluding auth headers from shared cache keys) were never captured in tests and therefore never reimplemented.

What this means for us is that tests are the strongest available verification mechanism for specified behaviors, but they cannot catch what they do not test for. The prose spec remains necessary for communicating intent, constraints, and the *spirit* of what you want, which are things that tests alone cannot express. The combination of prose specs + executable tests + iteration loops is more reliable than either alone.

5.4 Practical Guidance

For practitioners, this means:

Where possible, write the test before or alongside the spec. The test is the machine-verifiable part of the spec. The prose spec provides intent and constraints; the test provides the correctness contract. When specifying a task for an AI agent, include both: a prose description of what you want and why, and a set of tests (or at minimum, verification commands like `npm test`, `npm run lint`, type checks) that the output must pass. For autonomous-mode work, make test passage an explicit success criterion. The agent should be able to run verification itself and iterate on failures without human intervention, up to a defined iteration budget.

This approach works best for tasks with clearly defined outputs (code, APIs, data transformations). For subjective or creative tasks, prose specs remain the primary tool, but even there, you can specify structural constraints ("must be under 500 words," "must include exactly three examples") that serve as partial verification.

Part 6: Context Engineering

When building long-running agents or multi-step workflows, the spec expands beyond task description into managing what information the AI has access to at each step.

6.1 Context as Compilation

Google's Agent Development Kit team articulates the foundational insight: the naive pattern of appending everything into one giant prompt collapses under three pressures.[6] **Cost and latency spirals**, as cost and time-to-first-token grow with context size. **Signal degradation** ("lost in the middle") where research shows LLMs perform worst on information in the middle of their context window, attending most to the beginning and end. And **Context poisoning** when a hallucination enters the context from a previous step, it compounds across subsequent steps because the LLM context treats all information as equally authoritative.

The solution is a four-layer model that represents the consensus architecture across major agent frameworks in 2025–2026: **Working Context** (the computed view sent to the model for a specific call, which is ephemeral, and constructed for purpose), **Sessions** (structured event logs of the full trajectory, which are durable and complete), **Memory** (searchable knowledge queried on demand, which persists across sessions), and **Artifacts** (large objects stored by reference, and too large to fit in context).[6]

6.2 Practical Principles

Scope by default. Start with zero context and explicitly retrieve only what is needed. Do not start with "everything the agent might need."[5]

Stable prefixes for cache efficiency. The Manus team found that KV-cache hit rate is the single most important metric for production agents.[7] Because LLMs process tokens sequentially, even a single-token change in the prefix invalidates the cache from that point onward. Order context for maximum cache reuse: system instructions first (most stable), then memory (occasionally updated), then session state (frequently changing), then the current task (always changing).

The four operations of practical context engineering (from LangChain's Lance Martin): **Write**, that is, saving information outside the context window for later use (scratchpads, memory formation, lessons-learned logs). **Select**, or pulling relevant context into the window (RAG, targeted file reads). **Compress**, or retaining only the tokens needed (summarization, pruning stale outputs). **Isolate**, or splitting context across multiple agents or steps to prevent interference.[8] A spec for any multi-step AI workflow should describe how each of these operations will be applied.

6.3 Context Engineering Checklist

For multi-step or long-running AI workflows:

- ☐ Context layers are explicitly designed (working context / session / memory / artifacts)
- ☐ Each step has a defined context view (what information is available)
- ☐ Context ordering maximizes cache reuse (stable prefixes first)
- ☐ Summarization schema is defined (what must survive compression)
- ☐ Tool descriptions are clear, unambiguous, and non-overlapping

- ☐ Hallucination containment is addressed (how are errors in previous steps prevented from propagating?)
- ☐ Iteration budget and escalation protocol are defined (for autonomous loops)

Part 7: Anti-Patterns and Failure Modes

7.1 Specification Anti-Patterns

The Kitchen Sink Spec. Including every possible requirement in a single massive spec. This overloads the context window and introduces signal degradation. *Fix:* Right-size the spec to the task using the decision framework in Part 1.1.

The Aspirational Spec. Vague, untestable requirements like "make it intuitive" or "ensure high performance." These give the AI no actionable constraints. *Fix:* Apply the Vague Requirements Translator (Part 1.4) so that every qualitative requirement must be translated into a measurable behavior.

The Solution Spec. Prescribing implementation rather than the problem. "Use a React useState hook" instead of "The form must preserve user input across re-renders." This prevents the AI from finding better approaches. *Fix:* Separate functional requirements (what) from technical constraints (how). Prescribe implementation only when there is a specific architectural reason.

The Assumption Spec. Relying on information the AI does not have. "Follow our standard patterns" assumes the AI knows what those are. *Fix:* For every reference to existing patterns or conventions, include the specific pattern or provide a file path where the AI can find an example.

The No-Boundary Spec. Specifying what to do without specifying what *not* to do. This is the most common anti-pattern and the one with the highest cost. Without explicit boundaries, agents will modify files they shouldn't, add features you didn't request, and expand scope. *Fix:* Every spec must include a boundaries/non-goals section at least as detailed as the goals.

7.2 Workflow Anti-Patterns

Mode Rigidity. Staying in one AI mode regardless of the task. Using Autonomous mode for a one-line fix; using Assistive mode for a hundred-line boilerplate file. *Fix:* Before each task, ask: "Is this faster to do myself, to direct the AI, or to hand off entirely?" Switch modes freely.

Review Complacency. As AI-generated code volume increases, the psychological tendency is to review it less critically. This is particularly dangerous because AI-generated code tends to look plausible even when subtly wrong. *Fix:* Review rigor should scale with the risk of the change, not with the volume.

False Sense of Control. Even with elaborate SDD workflows, agents frequently did not follow all instructions.[4] Templates and workflows create a feeling of control without guaranteeing it. *Fix:* Trust but verify. After every agent execution, check specific constraints from your spec. Machine-verifiable constraints (tests, linters, type checks) are more reliable than manual review for catching violations.

7.3 Organizational Anti-Patterns

Spec-as-Bureaucracy. When specification becomes a box-checking exercise. Teams that require a full PRD for every one-line fix are using specification as bureaucracy, not engineering. *Fix:* Match specification effort to task risk and complexity.

No Feedback Loops. Writing specs, getting results, and moving on. Track your AI interaction failures in a simple log (date, task, failure, root cause, spec gap). After 10–20 entries, patterns will emerge, things you habitually forget to specify. Add these to your default template. This is the only reliable way to improve specification skill over time.

Tooling Lock-In. Adopting an SDD tool's opinionated workflow as the only way to specify work. *Fix:* Learn the principles (correctness contracts, decomposition, boundaries, tradeoff forcing) first. Evaluate tools by how well they implement those principles for your specific workflow.

Part 8: Reference — Full Templates and Evaluation

8.1 The Full PRD Template

[Project Name]: Product Requirements Document

1. What & Why
Problem: [What pain point exists]
Desired Outcome: [What success looks like]
Justification: [Why this is worth doing now]

2. Success Metrics
Metric	Current Baseline	Target	Measurement Method
[Metric 1]	[Value]	[Value]	[How to measure]
[Metric 2]	[Value]	[Value]	[How to measure]

3. User Stories
GIVEN [precondition]
WHEN [action]
THEN [observable result]

AND [additional observable result]

4. Acceptance Criteria
- [ ] [Binary-testable condition — "QA can verify pass/fail"]
- [ ] [Binary-testable condition]

5. Non-Goals (as detailed as the goals)
- [Explicit exclusion 1 — and WHY it is excluded]
- [Explicit exclusion 2 — and WHY it is excluded]

6. Technical Constraints
- Stack: [Languages, frameworks, libraries]
- Architecture: [Patterns, conventions, existing code to follow]
- Performance: [Budgets, limits]
- Security: [Requirements, policies]
- Integration: [External systems, APIs, data sources]

7. Open Questions
- [ ] [Known unknown 1 — who should answer this?]
- [ ] [Known unknown 2 — what information is needed?]

8. Uncertainty Policy
When uncertain: [Guess / Flag / Stop]
When forced to choose between [X] and [Y]: prefer [X/Y]

9. Verification
- Deterministic: [Tests, linters, type checks that must pass]
- Manual: [What a human reviewer should specifically check]

8.2 The Spec Self-Evaluator

Before executing a spec, evaluate it against this rubric. Each question maps to a specific failure mode.

Question	What It Catches
Is the problem stated before any solution?	Solution-first thinking
Are success metrics quantitative and binary-testable?	Aspirational metrics
Is the non-goals section at least as detailed as the goals?	Scope creep via omission

Are all constraints explicit (technology, patterns, performance, security)?	Under-constrained solution space
For each vague term, is there a measurable translation?	Ambiguous requirements
Does the spec describe verification steps?	Unverifiable output
Are uncertainty and error-handling policies stated?	Unpredictable AI behavior
Does the spec state which tradeoffs to favor?	Implicit tradeoff resolution
Is the abstraction level appropriate for the task size?	Over- or under-specification
Could a different person (or AI) unambiguously execute this spec?	Hidden assumptions

8.3 Output Evaluation: The Hierarchy of Verification

Once a spec has been executed, evaluation methods exist on a spectrum:

Deterministic checks (cheapest, most reliable). Does the code compile? Do tests pass? Does the linter report no new warnings? These are binary, automated, and should always be the first line of evaluation.

Specification compliance checks (moderate cost). Does the output meet each acceptance criterion? This requires manual review or an LLM-as-judge approach.

LLM-as-judge evaluation (moderate cost, variable reliability). Using a separate model to evaluate output against criteria. Well-prompted models can align with human judgment approximately 80% of the time. Caveats: LLM judges can exhibit verbosity bias, position bias, and self-preference.

Human review (most expensive, most reliable for judgment). The irreplaceable layer. Human review should focus on what automated systems cannot assess: security implications, architectural fit, compliance, and alignment with quality standards. Code review is not less important in the AI era, but it is *differently* important. Tools absorb the correctness burden; human review becomes the judgment layer.

The most common mistake is over-relying on human review for things deterministic checks could catch, or under-investing in human review for things automated checks cannot assess.

8.4 Sources Referenced

1. Gwin, C. (2025). "How Engineers Actually Work With AI: The Power Tool Spectrum." corygwin.com.

2. METR (2025). "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity." arXiv:2507.09089.
3. Liu, S. (2025). "Spec-driven development: Unpacking one of 2025's key new engineering practices." Thoughtworks.
4. Böckeler, B. (2025). "Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl." martinfowler.com.
5. Anthropic Engineering (2025). "Effective context engineering for AI agents." anthropic.com.
6. Google Developers Blog (2025). "Architecting efficient context-aware multi-agent framework for production."
7. Manus AI (2025). "Context Engineering for AI Agents: Lessons from Building Manus." manus.im.
8. Martin, L. (2025). "Context Engineering for Agents." LangChain blog.
9. Cloudflare (2026). "How we rebuilt Next.js with AI in one week." blog.cloudflare.com.
10. METR (2026). "We are Changing our Developer Productivity Experiment Design." metr.org.
11. Cybernews (2026). "Cloudflare's AI-built Next.js replacement hit by security flaws."

Further Reading:

- Park, H. (2026). "Using spec-driven development with Claude Code." Medium.
- "Chapter 5: Spec-Driven Development with Claude Code." Agent Factory.
- Denicola, D. (2025). "My Participation in the METR AI Productivity Study."

Closing Note

Cory Gwin was right: writing good specs is still hard, and there is no single reliable formula. The evidence compiled here demonstrates that specification is an evolving engineering discipline with identifiable principles, common failure modes, and teachable techniques. However it is not a solved problem.

The most honest assessment of where the field stands is that specification quality is the highest-leverage factor within the developer's control, executable tests are the strongest verification mechanism available, and the combination of prose specs + tests + iteration loops is the most promising pattern we have. But non-determinism remains a fundamental constraint. Prose specs do not guarantee compliance. Tests do not catch what they don't test for. The vinext project showed both sides of this: almost every line of code was written by AI, and every line passes the same quality gates you'd expect from human-written code, yet the project shipped with critical security flaws because undocumented behaviors were never captured in the test suite.

METR's February 2026 follow-up already shows evidence that the productivity picture is changing as tools improve and developers gain experience. The fundamental insigh, however,t will likely endure: the quality of AI output is bounded by the quality of the specification it receives. Improving your specification ability and pairing it with executable verification is the single highest-leverage investment you can make as an AI-assisted practitioner.

The models will eventually get better. The tools and harnesses will evolve. The non-determinism problem may or may not be solved. The specs and tests will still need to be written by someone who understands the problem. And that someone can only be you.